The two declarations of the function display shown in Code 1 and Code 2 look different from the programmer's perspective, but from the compiler's viewpoint, both are one and the same. The standard specifies that a parameter in the function declaration, which is of 'array of type(T)' shall be decayed to 'pointer to type(T)'.

Note 1: An array name in the declaration of a function parameter is treated by the compiler as a pointer to the first element of the array.

```
void display(int numbers[], int size);
void display(int numbers[5], int size);
```

In the code given above, all the function declarations are equivalent. One can call the function display with an array, with a pointer or with its actual argument. In all the three prototypes, the function parameter (numbers) will decay into the 'Pointer to the first element of an array'.

Example 2

Consider the example of a 2D array, to explain how it will decay into the 'Pointer to an array', when passed to the function. Let us start with a demo code as shown in Code 3:

```
1 #include <stdio.h>
2 #define ROW 3
3 #define COL 2
4
5 int main()
6 {
7     int array[ROW][COL];
8     populate(array);
9
10     //Some code here
11 }
12
13 void populate(int array[ROW][COL])
14 {
15         //Some code here
16 }
```

Code 3: *Code to demonstrate how 2D arrays are passed to functions*

From the example shown in Code 3, when the 2D array of type(T) is passed as a parameter to the function, it will decay into the 'Pointer to an array', as shown in Line 13 of Code 3. Code 3 shows a straightforward way of passing the 2D arrays to the function, which means declaring them exactly the same way as declared in the calling function.

```
1 #include <stdio.h>
2 #define ROW 3
3 #define COL 2
4
5 int main()
6 {
7     int array[ROW][COL];
8     populate(array);
9
10     //Some code here
11 }
12
13 void populate(int (*array)[COL])
14 {
15         //Some code here
16 }
```

Code 4: *Code to demonstrate how 2D arrays are passed to functions*

As far as we are concerned, with the equivalence between pointers and arrays, when arrays are passed as an argument to a function, what really gets passed is a pointer to the array's first element. When we declare a function that accepts an array as a parameter, the compiler simply compiles the function as if that parameter were a pointer, since a pointer is what it will actually receive.

In general, when multi-dimensional arrays are passed as a parameter to the function, what actually is passed is a pointer to the array's first element. Since the first element of a multi-dimensional array is another array, what gets passed to the function is a 'pointer to an array'.

We cannot receive a 2D array as shown in the demo Code 5, since compilers will generate code in such a way that functions will receive the 2D array as 'Pointer to an array' and not 'Pointer to Pointer'.

```
1 #include <stdio.h>
2 #define ROW 3
3 #define COL 2
4
5 int main()
6 {
7     int array[ROW][COL];
8     populate(array);
9
10     //Some code here
11 }
12
13 void populate(int **array) //Error
 14 {
 15         //Some code here
 16 }
```

Code 5: *Code to demonstrate that 2D arrays cannot be collected as pointer to pointer*

Now, consider Code 6 given below: